

Figure 15: Performance effect comparison of runtime statistics optimization for the interpretation path, AQPHub (DuckDB), and JIT path, JITHub (DuckDB), on JOB.

10 SUPPLEMENTAL MATERIAL

10.1 Performance Effect of Runtime Statistics Optimizations for Interpretation and JIT Separately

In this section, we compare the performance effect of runtime statistics optimization to interpreters and query compilers. As we described in Section 4, JITHub employs StatCol to collect runtime statistics after each sub-query's execution, then it applies `per-row guard`, `block skip`, and `existence pre-check` to filter out known unmatched rows based on runtime values (i.e., ranges). This idea, in theory, can be applied to database interpreters as well. We extend AQPHub with these runtime statistics optimizations and evaluate their effect. We find the same behavior for both JOB and DSB, so we only report the numbers on JOB here.

Overall. We observe execution speedup with runtime statistics optimizations for both AQPHub and JITHub, where JITHub improves more. This reveals that using runtime statistics to prevent unmatched rows before the actual probe accelerates the execution for both interpreters and query compilers. However, query compilers gain a greater improvement in this process.

Drilldown. We dig in to understand why JITHub benefits more than AQPHub from runtime statistics optimizations. First of all, JITHub uses a row-at-a-time execution mode, but AQPHub uses DuckDB's vectorized executor. By applying the runtime statistics optimizations, JITHub inserts the guard checking inside the loop, as shown in Algorithm 1. This makes it naturally well-suited for checking the range of probe keys row by row. Then it uses the LLVM toolchain to optimize it into a few native instructions. On the other hand, AQPHub's runtime statistics optimization requires a separate scalar loop before the vectorized execution, breaking the tight and compiler-friendly loops. Second, JITHub compiles the optimizations into native code, avoiding extra function calls in the hot path of the probe process, as we mentioned in Section 4.2. But AQPHub has the overhead of virtual function calls during runtime.

Conclusion. Figure 15 shows that both the AQPHub and JITHub benefit from runtime statistics optimizations. However, JITHub gains a greater improvement because the row-by-row execution mode of JITHub is naturally more suitable for row-level optimizations than the vectorized interpretation of AQPHub, and JITHub's compilation eliminates the virtual function call overhead.